Peer-to-Peer Networking and Applications
https://doi.org/10.1007/s12083-021-01268-2

A survey on security in consensus and smart contracts

Xuelian Cao[1] · **Jianhui Zhang**[1] · **Xuechen Wu**[1] · **Bo Liu**[1]

Received: 3 December 2020 / Accepted: 6 November 2021

Abstract
Blockchain technology has evolved from a cryptocurrency-exclusive technique for direct transactions among distrusting users (i.e., *Blockchain 1.0*), into a general programming paradigm for building decentralized applications (i.e., *Blockchain 2.0*). That greatly expands the application domain of *Blockchain 2.0* while importing much more security issues than *Blockchain 1.0*. Intensive research on the security of blockchain technology has been conducted, showing that security has become the most concerned topic in the blockchain realm, and consensus and smart contracts are the most vulnerable parts to be attacked. On account of this, we are concerned mainly in this review paper with security issues related to consensus and smart contracts. Different from previous surveys, this survey especially tries to provide a systematic and comprehensive view on the security of blockchain technology within consensus and smart contracts through the integral action-pathway from root causes, vulnerabilities, and attacks, to the consequences. Moreover, the proposed countermeasures to the security issues in consensus and smart contracts are also evaluated and discussed in a holistic manner. With our understanding of the surveyed methods, we believe that countermeasures should be proposed with full consideration of the causal relationships among causes, vulnerabilities, attacks, and consequences. We expect the current work can pave the way for a comprehensive understanding of how a security issue functions and where the undiscovered vulnerabilities and possible attacks hide, so as to systematically design the countermeasures.

Keywords Blockchain security · Consensus · Smart contracts · Security action-pathway

1 Introduction

The blockchain technology (BCT) is one of the most hyped buzzwords among both the academic and industrial communities of information and communication technologies (ICT) [1], financial technologies (FinTech) [2], economical sectors [3], manufacturing sectors [4], etc. When it initially appeared in 2008 as the underlying technology for the electronic cash system named Bitcoin [5], the BCT can be interpreted as a distributed publicly verifiable ledger, which allows mutually distrusting users to reach a consensus on the payment transaction between them, without any trusted third party. The blockchain is then characterized by [6]: (1) chain of hash-based timestamped blocks, (2) distributed storage and collective maintenance. With these characteristics, Bitcoin is enabled with traceability, tamper-resistant, direct transactions between any distrusting users, and high availability. The BCT applications represented by Bitcoin are limited to cryptocurrency for payment services [7]. We thus call the version of BCT the *Blockchain 1.0* as it was at this point used as the cryptocurrency-exclusive technique.

Encouraged by the success of Bitcoin, people start to exploit richer BCT applications. This requirement is filled by introducing a new part, namely, *smart contracts* [8] into BCT. A *smart contract* refers to a shareable program which encapsulates predefined state, transition rules, trigger conditions, and corresponding response rules [9]. After the deployment, the smart contract autonomously runs on the blockchain. Interaction of smart contracts leads to a new paradigm of decentralized applications (DApps) [10], which enables wider application domains of medicine [11], economics [12], internet of things (IoT) [13–15], digital assets [16], and smart cites [17–19]. We call the BCT with smart contracts the *Blockchain 2.0*. The representative of this version is Ethereum.

✉ Bo Liu
liubocq@swu.edu.cn

Xuelian Cao
xuelian7@email.swu.edu.cn

[1] RISE, School of Computer and Information Science, Southwest University, Chongqing 400715, China

Published online: 12 January 2022

We are now in the era of *Blockchain 2.0*. As a general low-cost trust enabling technique and a new information infrastructure to develop and operate DApps, the new BCT has been regarded as the cornerstone of forthcoming *Digital Economy Era* [20]. Moreover, the BCT can also be applied in real-time IoT applications, providing trust in distributed robotic systems running on embedded hardware without the need for certification authorities [21, 22]. For example, Jiang et al. [23] proposed a permissionless Byzantine consensus protocol in wireless networks for time-critical IoT application scenarios. However, the additional functionalities of the new BCT bring increasing vulnerabilities surfaces that are exposed to a large number of security attacks, especially on consensus protocols and smart contracts, thereby leading to considerable losses in real-world blockchain platforms [24]. For example, the DAO attack on smart contracts in 2016 has stolen about $60 million through recursive calling vulnerability [25]. Parity Multisig Wallet was reported in 2017 to have experienced a loss of about $30 million due to the vulnerability in the wallet contract[1]. MyEtherWallet was also reported in 2018 to be stolen about $17 million due to the BGP and DNS hijacking attack[2]. The Ethereum Classic suffered three 51% attacks on August 2020 alone, resulting in considerable losses[3]. The statistics we have conducted shows that security-related attacks have caused more than $136.07 million economic losses[4].

As shown in Fig. 1, security has become the fastest increasing concern that ranks first among all BCT-related topics[5]. Based on the collected data[6] in Fig. 2, we further find that the consensus (including consensus protocols and peer-to-peer networks) and smart contracts are particularly vulnerable in both *Blockchain 1.0* and *2.0*. To this end, more special efforts should be invested into the security issues in consensus and smart contracts. Herein, we are mainly concerned with the state-of-the-art of the related security issues and countermeasures.

Over the past few years, intensive research works have been conducted on blockchain security issues. Accordingly,

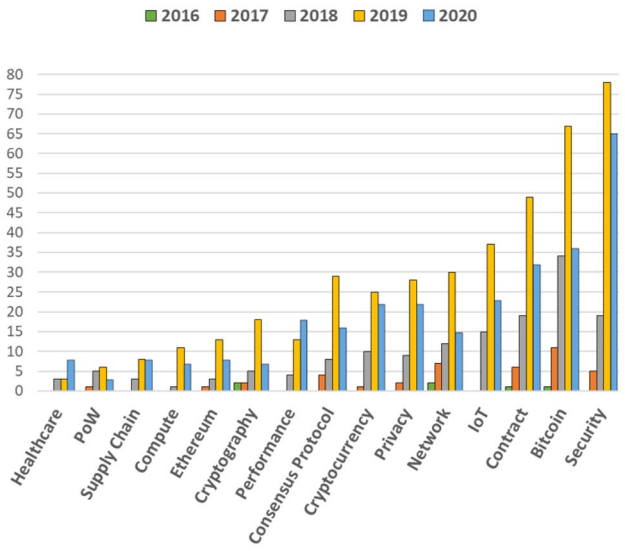


Fig. 1 Rank of topics in BCT surveys

there are also numerous surveys reporting the attacks, causes, countermeasures, etc., upon different technique parts of blockchain on various blockchain platforms [26–29]. We collect the most closely related surveys among them to the current paper, which were published in the last three years. Then, we make a comparative discussion from the perspectives of specific platform and technique focuses and coverage of security concerns (including the cause, vulnerability, attack, and countermeasure), as shown in Table 1.

- *Specific platform focuses:* Bitcoin and Ethereum, as the representative platforms of blockchain 1.0 and blockchain 2.0 respectively, have suffered numerous attacks and drawn a tremendous amount of eyes on the research of platform-specific security. For example, Zaghloul et al. [30] investigate the security threats that Bitcoin suffers, while the Ethereum-specific security issues are also surveyed by [26, 29, 31, 32]. As an increasing number of blockchain platforms are developed and used by both industrial and academic communities, the platform-specific surveys on security issues appear not so wide in coverage and comprehensive of analysis. Moreover, a lot of attacks are featured in a cross-platform manner (e.g., BGP hijacking attacks on both Bitcoin and Ethereum [28]). As a result, a platform-specific analysis of attacks tends to be one-sided, while the corresponding countermeasures could also be evaluated without a systematic view. In light of this, a significant amount of research on general platform-oriented security issues are carried out, and the corresponding surveys are also conducted [27, 33–36]. They analyse the commonness and characteristics of attacks on various blockchain platforms while evaluate the countermeasures holistically and compre-

[1] https://blog.openzeppelin.com/on-the-parity-wallet-multisig-hack-405a8c12e8f7/

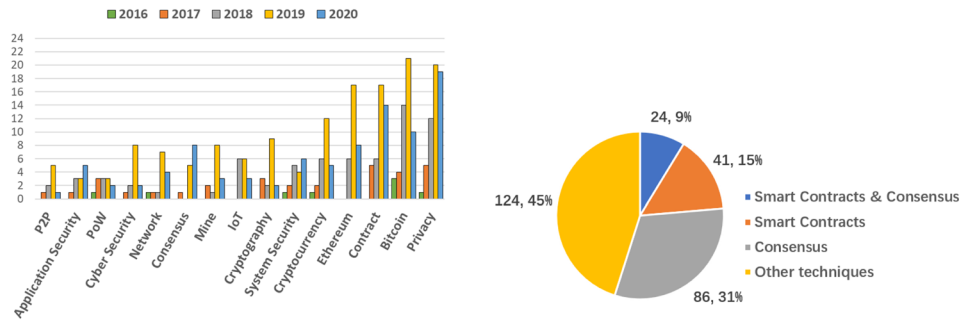
[2] https://news.bitcoin.com/myetherwallet-servers-are-hijacked-in-dns-attack/

[3] https://www.coindesk.com/crypto-51-attacks-etc

[4] https://hacked.slowmist.io/

[5] Details of the review process: https://docs.google.com/document/d/12iFGkrcprvD1Rzm6XMbQAMUnHMMzEePctfxax19ORoc?usp=sharing
References: https://docs.google.com/spreadsheets/d/1KuTZU-tTdd0UlRe9f9l3dcYoZFHEaabnSaYujK7DovU?usp=sharing

[6] Security-related references: https://docs.google.com/spreadsheets/d/1bJzbcVn4aQ1AsWCy1K_klSI74g772m1cKyI1qrdHKSI?usp=sharing



(a) Rank of security-related topics

(b) Proportion of various blockchain techniques

Fig. 2 Statistics on various blockchain security issues

hensively. The general platform is also what the current survey focuses on.

- *Specific technique focuses:* The collected data and statistics show that smart contracts and consensus are the key concerns in blockchain security issues. Kolb et al. [31] discuss the Ethereum security concerns, and especially focus on the challenges in Solidity (a smart contract programming language) programming and programs, as well as the consensus algorithms and protocols. Wang et al. [32] review existing research literatures on the security of Ethereum smart contracts, categorize security challenges into abnormal contracts, program vulnerability, and unsafe external data. Kim et al. [37] survey existing research works on smart contracts from the perspective of static analysis for vulnerability detection, static analysis for program correctness, and dynamic analysis. Tolmach et al. [38] review the recent advances in the application of formal methods to analyse smart contracts, with a focus on various formalisms supporting the specification and verification of the domain-specific requirements. According to the aforementioned surveys, we obtain a thorough understanding of the security issues regarding the smart contracts and consensus protocols. However, the security of smart contracts and consensus should not only be restricted to programs and protocols. Increasing evidence shows that other underlying technique parts of smart contracts and consensus should not be neglected when discussing blockchain security issues, particularly P2P networks [39]. To the best of our knowledge, few surveys have conducted a holistic review on the systematic security of P2P networks, consensus protocols, and smart contracts.
- *Coverage of security concerns:* A systematic survey on blockchain security should cover the cause, vulnerability, attack, countermeasure, as well as the relationships between them [26, 29, 40]. In recent surveys related to

Table 1 Existing surveys on the security of blockchain

Ref	Year	Platform	Technique	Coverage of Security Concerns			
				Cause	Vulnerability	Attack	Counter measure
[33]	2019	General	Networks, SC, Crypto		✓	✓	✓
[36]	2019	General	–				✓
[26]	2020	Ethereum	–	✓	✓	✓	✓
[27]	2020	General	–		✓	✓	✓
[37]	2020	General	SC		✓		
[31]	2020	Ethereum	SC				✓
[35]	2020	General	–			✓	✓
[30]	2020	Bitcoin	Networks, CP			✓	✓
[39]	2021	General	Networks			✓	
[28]	2021	General	–		✓	✓	✓
[34]	2021	General	–		✓		✓
[29]	2021	Ethereum	SC	✓	✓	✓	✓
[38]	2021	General	SC	✓	✓		✓
[32]	2021	Ethereum	SC		✓		
This work		General	–	✓	✓	✓	✓

the security of the BCT, Wang et al. [32] review the exploitable vulnerabilities suffered by Ethereum smart contracts, Dotan et al. [39] survey security issues of the blockchain with a focus on network-layer attacks, Saad et al. [35] explore attacks and ongoing countermeasures in terms of distributed system architecture, P2P network, and application-oriented, Zaghloul et al. [30] analyse the attacks suffered by Bitcoin and investigate some possible countermeasures. These surveys focus on a specific blockchain platform or some technique parts of the BCT and conduct the security analysis with different coverages of security concerns. Although many other surveys provide an understanding of blockchain security from a relatively broader view [27, 28, 33, 34], they also underperform in the coverage of security concerns. By contrast, Chen et al. [26] and Samreen et al. [29] provide respectively systematic understandings of these security concerns. However, they are all limited to the discussion of Ethereum smart contracts. The aforementioned surveys uncover some vulnerabilities or attacks and summarize some state-of-the-art countermeasures against security issues of the BCT. However, an analysis for the root cause of the vulnerability is quite few. Thus, there is still a lack of an in-depth analysis of the BCT security, which analyses related security concerns without a focus on a specific blockchain platform or underlying technique.

In general, it still demands a comprehensive and systematic survey that discusses the security issues regarding general blockchain platforms. Such a survey includes the whole action-pathway within both consensus and smart contracts from the root causes, vulnerabilities, attacks, to the consequences. Furthermore, the survey should conduct analysis on the countermeasures in a holistic manner and provide integral suggestions on securing consensus and smart contracts. Different from the previous platform-specific and technique-specific surveys, we try to present this survey paper as:

- a systematic understanding of the security action-pathway: how root causes within various layers of blockchain result in consensus and smart contracts vulnerabilities, and how vulnerabilities lead to a special category of attacks, thereby resulting in consequences;
- a comprehensive evaluation of the proposed countermeasures which is not only on an attack itself but also on the whole security pathway underlying;
- a guide to design integral countermeasures based on the presented security action-pathway.

To the best of our knowledge, the same work as above is still unavailable in previous studies.

The remainder of this paper is organized as follows: In Sect. 2, we present an overview of BCT where the consensus and smart contracts are specially introduced. We systematize the security action-pathway of consensus in Sect. 3 and that of smart contracts in Sect. 4. Sect. 5 is a holistic discussion of findings we have learned from existing research works and future research directions. Finally, We conclude this survey in Sect. 6.

2 Preliminaries

For the technical discussion in the following sections, we would like to introduce underlying techniques and security requirements in the layer of consensus and smart contracts, respectively.

2.1 Consensus

In the context of the BCT, the right to generate a new block is in the hand of multiple nodes instead of a central node. Furthermore, all nodes in the Peer-to-Peer (P2P) network have the option to append the newly generated block to their local copy of the blockchain or to ignore it. The consensus protocol is thus utilized to seek the majority of the network to reach a consensus upon the newly generated block (i.e., an update to the blockchain). Consequently, the underlying techniques involved in the consensus process include P2P networks and consensus protocols.

P2P networks: The P2P network is a distributed application architecture to allocate tasks and workloads among peers [41], and a networking technique or network communication form originated from the P2P computing model. In the P2P network, there is no central authoritative node, and all nodes can undertake the following tasks: discovery of new nodes, network routing, and verification of data disseminated in the entire network. Based on this characteristic, the BCT utilizes the P2P network to concatenate nodes in different physical locations to participate in the consensus process. Depending on whether the network is decentralized and addresses of nodes are structural, P2P networks can be divided into four categories: centralized P2P networks, decentralized unstructured P2P networks, decentralized structured P2P networks, and semi-distributed P2P networks. Wherein the latter three are adopted by practical blockchain platforms, taking Bitcoin [42], Ethereum [43], and Hyperledger Fabric as the representation, respectively.

Consensus protocols: In the field of distributed computing, consensus protocols are proposed to solve the consistency problem suffered by distributed systems, namely, how to ensure that all nodes in a distributed cluster have

identical data and can agree on a proposal [44]. Inspired by the research related to distributed computing, the BCT explores the use of consensus protocols in maintaining the consistency of blockchain copies at all nodes, with an expectation of stable and reliable growth of the blockchain. Since the introduction of proof of work (PoW) consensus protocol to Bitcoin [45], a large number of consensus protocols have been proposed, e.g., proof of stake (PoS) and Practical Byzantine Fault Tolerance (PBFT) [46]. Even if using different consensus protocols in different blockchain platforms, the consensus process generally involves three cyclic stages: (1) the selection of the node which is responsible for generating the next block, (2) the generation and dissemination of the newly generated block, (3) the verification and addition of received blocks. The difference between different consensus protocols mainly lies in the selection of such special node and the sequence of the first two stages.

2.1.1 Security requirements

Generally, the consensus in the BCT should provide two fundamental security requirements, namely, consistency and fault-tolerance [31, 47].

Consistency: Consistency in the context of the BCT refers to the requirement that all nodes in the P2P network have the same copy of the blockchain [47]. Wherein, the newly generated block is disseminated and verified by all nodes, stored linearly and chronologically on the copy of the blockchain at each node if it passes the verification. All copies of the blockchain will eventually get consistent through verification rules prescribed by the consensus protocol. Arguably, the consistency depends on the consensus protocol adopted in the specific implementation of the blockchain. For example, the PoW-based blockchain (i.e., the blockchain using PoW consensus protocol) provides a weak consistency which means that the copy of the blockchain at each node gets consistent eventually, even though some read/write requests to the blockchain may return stale data [36]. The BFT-based blockchain (i.e., the blockchain using a series of BFT consensus protocols) provides a strong consistency which means that all nodes have the same copy of the blockchain at the same time.

Fault-tolerance: The fault-tolerance means that the consensus on blocks can still be reached in the case of communication malfunctions or deviant behaviors of some nodes, which are commonly seen in the real-world setting. In the context of the BCT, this requirement of consensus is provided by the distributed structure of the P2P network, the consensus protocol, and the chained data structure of the blockchain. The fault-tolerance of consensus aims to guarantee the tamper-proof consensus results and stable growth of the blockchain even if there exist unexpected cases mentioned before. For example, PBFT consensus protocol allows no more than a third of failure nodes such that failure of such nodes does not disrupt the growth of the blockchain [46, 47]. Due to the need for recalculating the hash of a block and all subsequent blocks, in this case, it is extremely difficult for an adversary to make successful tampering with the block in the blockchain.

2.2 Smart contracts

The concept of smart contracts is initially proposed by Nick Szabo, who defined it as "a set of promises specified in digital form, including protocols within which the parties perform on these promise" [48]. By encoding the terms of the agreement as computer-readable codes, smart contracts enable multiple parties to execute contracts encompassing the terms of the agreement without the need for a trusted third party. However, smart contracts have not been widely used due to a lack of the secure and reliable execution environment, which guarantees the correct execution of contract terms without a trusted third party.

With the advent of the BCT, smart contracts have been introduced to represent autonomous computer programs that are deployed on the blockchain and executed by multiple nodes in the P2P network. The BCT provides a secure and reliable execution environment for smart contracts with the characteristics of trust-free and tamper-proof. Moreover, smart contracts enable the underlying data of the blockchain programmable and expand the application scenarios of the BCT from initial cryptocurrency to a broader scope.

2.2.1 Security requirements

Smart contracts have been widely applied to a number of scenarios because of the ability to automate the execution of pre-defined rules [38]. The well-known examples include cryptocurrency exchanges, crowdfunding campaigns, social games (e.g., auctions, voting schemes, lotteries), etc. Commonly, smart contracts in such scenarios require several fundamental security requirements that include call integrity, atomicity, and independence.

Call integrity: There exists an issue in the execution of smart contracts that the contract behavior may depend on an external call [49–52], which may not be in the initial design of the developer. Specifically, a contract can call another contract's functions and wait for returned values or call another contract that re-enters this calling contract. In such a case, the contract's control flow should not be influenced by attackers to prevent malicious attacks that could sabotage the correct execution of contracts.

Atomicity: If a part of the functions in the callee contract fails to execute, then the entire execution process fails and the state should be left unchanged [38, 51, 53, 54]. Taking

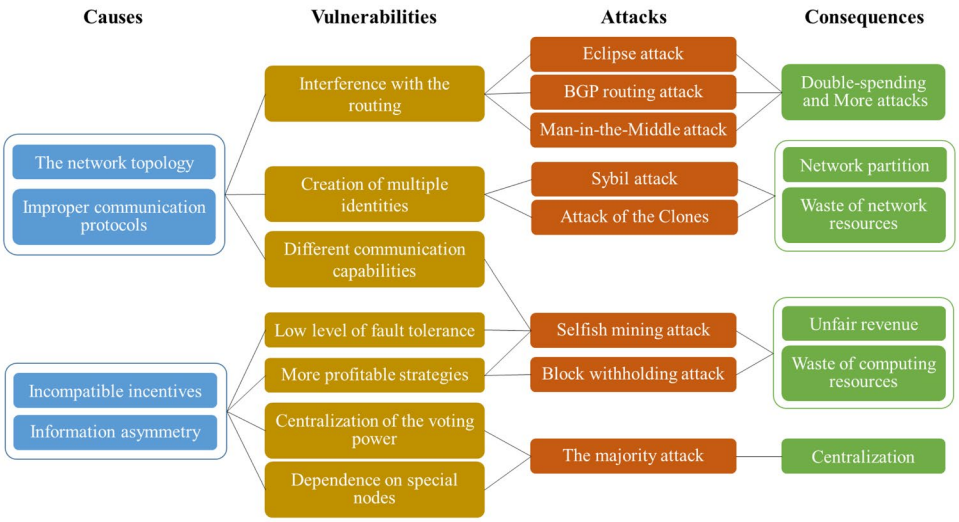


Fig. 3 Action-pathway of causes-vulnerabilities-attacks-consequences on consensus

Ethereum as an example, the callee contract should terminate when an exception (e.g., out-of-gas error, exceeding call-stack's depth limit [53]) occurs to avoid serious consequences caused by a violation of the atomicity requirement. For instance, an exception may cause considerable losses of money when the call involves numerous assets.

Independence: Ideally, the execution of smart contracts should be independent of parameters that are susceptible to the external environment [49–51, 55]. However, some smart contracts may include a timestamp-dependent function, which can be affected by different timestamps at nodes. The order of transactions that change a contract's state can also be affected by the nodes' behaviors of paying a higher transaction fee. Consequently, nodes can exert an influence on the execution of contracts by influencing such parameters.

In this work, we mainly focus on the fundamental security requirements instead of providing a comprehensive description. Apart from the security requirements mentioned above, actually, there is support for additional security guarantees of smart contracts in some specific scenarios, which include liquidity [56, 57], proper access control for safety-critical operations [57, 58], and reasonable resource consumption [59–61].

3 Insecure consensus

Security requirements presented in Sect. 2.1.1 indicate that P2P networks and consensus protocols play an important role in the security of consensus. Deficiencies in the design and implementation of P2P networks and consensus protocols can violate the security requirements mentioned above, resulting in security vulnerabilities that further be exploited by malicious attackers to perform attacks. In this section, we first discuss the vulnerabilities in these two aspects and then enumerate several well-known attacks related to the mentioned vulnerabilities (Sect. 3.1). An in-depth analysis of these vulnerabilities and current countermeasures can be seen in Sect. 3.2 and Sect. 3.3, respectively. Additionally, we present an action-pathway of causes-vulnerabilities-attacks-consequences related to consensus in Fig. 3.

3.1 Vulnerabilities and attacks

3.1.1 Vulnerabilities in P2P networks

Vulnerabilities in P2P networks mainly include the creation of multiple identities, interference with the routing, and different communication capabilities.

Creation of multiple identities (VC1): In the P2P network, each node is identified by a unique ID (i.e., public key). However, an attacker could exploit multiple identities forged on the same machine to participate in the P2P network and jointly work for the attacker. Taking the *Sybil attack* [31, 62] as an example, an attacker obtains the routing information about the remaining nodes in the P2P network by creating multiple identities. Then, the attacker misdirects the network routing of victim nodes and consumes network resources simultaneously. The *Sybil attack* is commonly discovered in the permissionless blockchain, which has no requirement for an authenticated environment [31], while the permissioned blockchain is also vulnerable to this attack. The high requirement for communication between nodes leads to a limitation on the size of the network [63]. Hence, the *Sybil attack* is likely to happen in smaller sizes of such a network. There exists an attack named *attack of the Clones* that is very similar to the *Sybil attack* [64]. In such an attack, the attacker duplicates multiple node instances and then partitions the network with one cloned instance in each partition. Therefore, double-spending and conflicting transactions might be accepted in separate partitions.

Interference with the routing (VC2): This vulnerability allows an attacker to isolate victim nodes from the P2P network by hijacking the connections of victim nodes or utilizing the lack of validation rules in the routing protocol. Once the isolation is successful, the attacker can control the routing of victim nodes and further affect these nodes' normal communication with the remaining nodes. Well-known attacks that exploit *VC2* include the *eclipse attack* [65], the *Border Gateway Protocol (BGP) routing attack*, etc. In an *eclipse attack*, the attacker can control the routing information of the victim node by inserting connections to malicious nodes into the victim node's routing table. Heilman et al. [65] indicate that an attacker with 32 distinct identities can perform the *eclipse attack* with a high probability of success (over 85%). Several studies subsequently show that the *eclipse attack* can reduce the difficulty of other attacks, such as '*N*-confirmation' double-spending [66], *selfish mining* [67]. Due to the deficiency that transactions or blocks are disseminated as plaintext and the validity of routes is unchecked in BGP [68], moreover, an attacker also can violate a group of nodes and further separate the network into multiple partitions by launching a *BGP routing attack*. Similarly, the *Man-in-the-Middle* (MitM) attack [69] proposed against Ethereum illustrates that an attacker can manipulate the communication delay between nodes by hijacking a BGP route. Once the attacker is in control of the communication delay, they can launch other attacks, such as the *Balance attack* [70].

Different communication capabilities (VC3): The network connectivity differs significantly across different network regions. Nodes residing in a highly connected region can disseminate blocks faster than those in a less connected region [71–73]. In addition, the difference in the number of transactions that nodes encapsulate into a block causes different communication delays as well [72, 74]. Consequently, different nodes have different communication capabilities, namely, it takes different delays for nodes to disseminate their blocks to the entire network. As a result, attackers can further exploit this vulnerability to launch an attack (e.g., *selfish mining* [73]) and thus undermine the security of consensus.

3.1.2 Vulnerabilities in consensus protocols

Vulnerabilities in consensus protocols mainly stem from the discrepancy between the fault-tolerant ability of consensus protocols and the changing environment. Specifically include the followings:

Centralization of the voting power (VC4): In the early stage of several blockchain platforms being proposed, each node can participate in a competition for the right of generating blocks by its voting power (e.g., the computing power in PoW or the stake in PoS). As more nodes participate in the competition, it is difficult for a solo node to control the majority of voting power in the entire network, which is considered as a security guarantee for the consensus [45]. However, multiple nodes can assemble their voting power and compete with other nodes in a form of a consortium (i.e., "pool") [67, 75]. In this way, a pool is prone to control the majority of voting power compared to a solo node and launches a *majority attack* (i.e., *51% attack* [35]), which can create an alternative copy of the blockchain with false history and enable more attacks further.

Nowadays, pools are very prevalent in public blockchain platforms and play a dominant role in block generation, causing the network to become centred gradually [76]. Taking Bitcoin and Ethereum as examples, the computing power of the top five pools exceeds 71.15% of the total computing power in the Bitcoin's network and the proportion is 83.8% in Ethereum[7]. Some pools are apt to reach the threshold of launching the *majority attack*. For example, a pool called GHash.IO in Bitcoin had once taken up over 50% of the computing power on one day in June 2014[8].

More profitable strategies (VC5): This vulnerability commonly occurs in which the honest strategy is regarded as the most profitable strategy for each node that participates in the consensus process [67, 75, 77–79]. The honest strategy refers to that nodes disseminate the newly generated block to other nodes as soon as the block generation is completed, and thus obtain the block reward matched with its proportion of voting power in the entire network.

However, a number of strategies deviating from the honest strategy are arguably shown to be more profitable [67, 75, 77]. Well-known examples of these strategies are selfish mining (SM) and block withholding (BWH). In the case of the SM strategy, attackers hide the newly generated block temporarily to seek more revenue more than their fair share, leading to a waste of computing resources and gradual centralization of the network [67]. With the BWH strategy, attackers delivery part of the participants (i.e., nodes) to victim pools and obtain revenue without any contribution, resulting in wastage as well [75]. Such strategies or combination of them would help attackers launch other attacks (e.g., *eclipse attack* [80]) and significantly obtain more revenue [81–83]. However, it is difficult to detect such strategies and even come up with feasible countermeasures. Currently, the vulnerability resulted from these strategies remains to be an open research problem.

Dependence on special nodes (VC6): This vulnerability is commonly discovered in a certain kind of blockchain that groups the nodes in the network with different roles. In such

[7] https://btc.com/stats/pool

[8] https://www.coindesk.com/bitcoin-mining-detente-ghash-io-51-issue

a blockchain, the security of the consensus depends on some special nodes to some extent, such as the primary which is responsible for processing transactions and blocks in PBFT-based blockchain [35], the witness which issues and orders transactions in Byteball [84] and HarmonyDAG[9], and the double-spending prevention (DSP) nodes which are used to address the double-spending problem efficiently in COTI [85]. This vulnerability may happen when behaviors of these special nodes deviate from what the consensus protocol prescribes. For example, the primary rearranges the order of transactions to delay the the process of block verification and block generation.

Low level of fault tolerance (VC7): There exists a vulnerability in most consensus protocols which are inherently a low level of fault tolerance or become a low level of fault tolerance as a result of being affected. For example, PBFT can tolerate up to 33% malicious replicas (a.k.a, byzantine nodes) [35], PoW or PoS are originally designed to tolerate up to 50% byzantine nodes. While this threshold could be decreased due to the centralization of the voting power and more profitable strategies [67, 75, 77, 78]. Arguably, a lower level of fault tolerance can further reduce the conditions required for successful attacks, leading to the insecure consensus.

3.2 Analysis of root causes

The aforementioned vulnerabilities and attacks show that security requirements presented in Sect. 2.1.1 cannot be well guaranteed in practice. This is because these security requirements are based on several implicit and idealistic assumptions. Firstly, they assume that all nodes have the same communication capability, which means nodes disseminate transactions and blocks throughout the entire network equally fast [73]. Secondly, the consensus process with the introduced incentive mechanism in some blockchain platforms is thought to be incentive-compatible [86], namely, nodes are incentivized to comply with the consensus protocol and thus obtain a block reward matched with their contribution. Thirdly, they argue that it is extremely difficult for nodes to reach the fault-tolerant threshold [35].

However, P2P networks do not work as described in the first assumption due to the network topology and improper communication protocols [26, 65, 68]. Specifically, attackers can obtain the routing information from the network topology and then hinder the normal communication of victim nodes by creating multiple identities or interfering with the routing of victim nodes [65, 68]. In addition, the network topology in a real-world setting also shows that nodes have different capabilities [73]. Meanwhile, improper communication protocols can affect the dissemination of transactions or blocks, such as discarding the transactions or causing a delay to the dissemination [26].

Apart from the communication capability of P2P networks, nodes involved in the consensus process also have a significant impact on the security of consensus. For example, the situation that the majority of nodes comply with the consensus protocol is regarded as a security guarantee [45]. In practice, the block reward that nodes can obtain is reduced and unstable due to the incentive mechanism and increasingly intense competition, propelling nodes to maximize their profits in other ways (e.g., pools or more profitable strategies). Hence, the consensus process of the BCT is not incentive-compatible under the existing incentive mechanisms [67, 75, 77–79].

The incentive-incompatibility of the consensus process leads to the emergence of pools and more profitable strategies. While the information asymmetry between nodes provides a convenience for launching malicious attacks [87, 88]. Information asymmetry refers to that nodes have different knowledge of the consensus process, such as the nodes' unawareness of the voting-power distribution of the entire network and hidden blocks of nodes using SM strategy. Typically, the party with more information holds a competitive advantage over the other nodes. The majority of more profitable strategies are pretty good examples that take full advantage of information asymmetry, e.g., SM, BWH. Wherein, malicious attackers exploit the information asymmetry to lower the fault tolerance of consensus protocols and thus increase the possibility of launching attacks successfully.

The root causes mentioned above result in lots of vulnerabilities and attacks, which undermine the consensus security of the BCT. However, there are still no efficient ways to detect and counter against the majority of these vulnerabilities to date.

3.3 Countermeasures

In this subsection, we analyse the latest progress of countermeasures related to vulnerabilities and attacks in P2P networks (VA-PN), as well as vulnerabilities and attacks in consensus protocols (VA-CP) respectively. A summary of countermeasures can be seen in Table 2.

3.3.1 Countermeasures against the VA-PN

In the past few years, a number of countermeasures related to the VA-PN have been proposed. Wherein, the vulnerability that creates multiple identities can be eliminated by restricting the node generation process, e.g., increasing an authentication for nodes or redefining the node ID with a combination of IP address and public key [26]. However, some of these countermeasures have not been adopted by developers due to the negative impact on the usability.

[9] http://www.harmonydag.com/

Table 2 Countermeasures against the VA-PN and VA-CP

Vulnerabilities	Countermeasures	**Refs**
VC1	restricting the process of nodes' generation	[26]
VC2	modifying the connection setting	[65, 89]
	increasing the anomaly detection	[90]
VC3	building relay networks	[91]
VC4	lowering the reward variance	[92]
	monitoring the participants	[93]
	selecting the closed strategies	[94]
VC5	detecting and defensing	[95–97]
VC6	optimizating consensus protocols	[67, 98–100]
VC7		

Regarding the interference with the routing, several research works concentrate on modifying the connection setting between nodes or increasing the anomaly detection. For example, Heilman et al. [65] propose to improve randomness in choosing nodes to be connected, Alangot et al. [90] utilize out-of-band gossip networks to detect the *eclipse attack* by exchanging their viewpoints on the blockchain (i.e., block headers) with lightweight clients, and Tran et al. [89] propose to protect the underlying network from the *eclipse attack* by increasing the number of nodes' outing connections. However, some research works may exert a negative impact on the throughput of the blockchain.

To mitigate the impact caused by different communication capabilities of nodes, several relay networks are proposed successively, e.g., FIBRE[10], Falcon[11], and SABRE [91]. Such relay networks can decrease the dissemination delay between nodes and provide a security guarantee for the dissemination process. However, these relay networks are required to provide additional connections between nodes that have special needs and need to run alongside the original network of the blockchain.

3.3.2 Countermeasures against the VA-CP

In consideration of vulnerabilities and attacks of existing consensus protocols, there exists a great need for secure consensus protocols, which remains an ongoing research area. Currently, the following efforts have been made in this direction.

Mitigation of the centralization: The centralization of the voting power makes the consensus vulnerable to lots of attacks (e.g., the *majority attack*). Given this, researchers have proposed several countermeasures to prevent the consensus from the centralization during the past few years. For example, Luu et al. [92] propose the SMARTPOOL that lowers the reward variance of miners and gives the right of selecting transactions back to miners, such that it decentralizes the mining pools. Dey et al. [93] propose an approach of using techniques of *Game Theory* and *Machine Learning* to monitor miners to detect the possible collaboration and take further action. However, this work is still in progress and only aims at the PoW consensus protocol. Wang et al. [94] explore the pools' selection on whether to be open or closed and discover that the closed strategy can protect the pools from attacks (e.g., *selfish mining*) and avoid the formation of large pools. Whereas, most of these are theoretical research and are rarely adopted in practical blockchain projects.

Detection and defenses: Regarding attacks resulted from some more profitable strategies, there are several ways to detect them in the practical blockchain. For example, Saad et al. [97] utilize the difference between the expected confirmation height of transactions[12] and the actual block height to detect *selfish mining*. Wherein, the greater the length of the private chain of nodes using the SM, the higher the value of the difference will be. An algorithm is also proposed to reject the private chain if such an attack is detected. Similarly, Chicarino et al. [95] detect *selfish mining* and *stalker attack* by observing the number and height of forks, analysis of the simulation results shows that an attack could already exist if the height of a fork is equal to or greater than 2. To consider a richer strategy space, Hou et al. [96] propose the SquirRL which is a framework using Deep Reinforcement Learning (DRL) to identify more profitable strategies. To the best of our knowledge, however, existing defenses proposed for these attacks are usually difficult to deploy in practice or remain imperfect [101, 102]. Moreover, related research works are far less than the research of analysis or detection on these attacks, and this is still an area in need of more concern.

Optimization on consensus protocols: Apart from the detection and defenses of malicious attacks, some solutions usually modify consensus protocols or put forward new consensus protocols, hoping to reduce the attacker's profitability or losses of the honest parties. Modification on consensus protocols mainly involves the fork-resolving policies which aim to reduce the probability that the honest parties work on the attacker's chain during a block race against the same height. Such fork-resolving policies include policies of uniform tie-breaking [67], largest-fee and smallest-hash tie-breaking [100], and unpredictable deterministic tie-breaking [99]. Except for this kind of modification, several new consensus protocols are proposed to address security

[10] http://bitcoinfibre.org/

[11] https://www.falcon-net.org/

[12] The index number of a future block in which the transaction is likely to be mined.

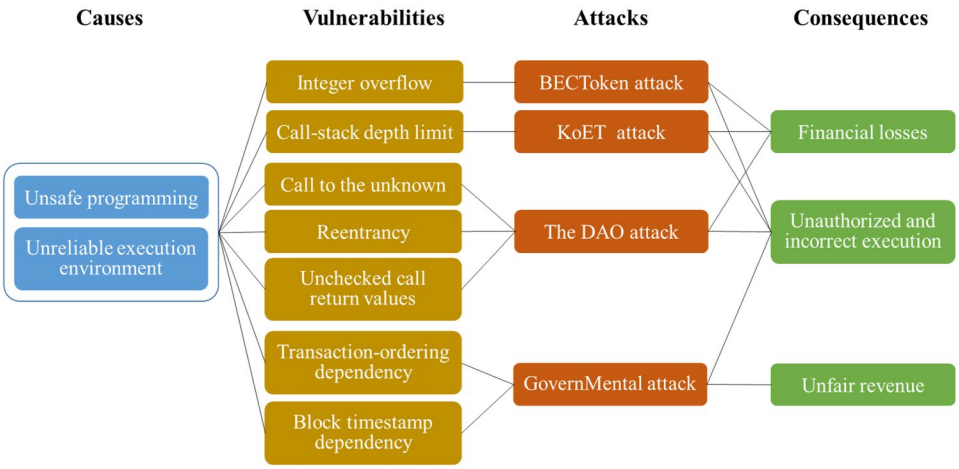


Fig. 4 Action-pathway of causes-vulnerabilities-attacks-consequences on smart contracts

issues of the consensus. For example, the Tendermint which is a high-performance consensus protocol can be resistant to one third of byzantine nodes at most and has high-level protection against *double-spending attack* [98].

Optimization on incentive mechanisms: Optimization on incentive mechanisms usually modify the reward distribution policy to reduce the revenue variance, and thereby make it no sense to gain more by launching attacks. These designs reward nodes that contribute to stabilize the blockchain [103] or reward most of the nodes selectively. Most nodes can thus receive a fraction of the block reward after modifying incentive mechanisms [102]. In addition, some punishment rules are proposed for malicious behaviors of nodes [104, 105].

The consensus process in most blockchains can be modeled as a game in which consensus protocols and incentive mechanisms jointly affect strategies and revenues of all players. Therefore, there is still a need for a joint optimization on consensus protocols and incentive mechanisms, although the independent optimization of consensus protocols or incentive mechanisms can partly mitigate the impact of malicious attacks. However, research work in this direction is quite few and needs more attention in the future.

4 Insecure smart contracts

Similar to Sect. 3, we discuss several well-known vulnerabilities and related attacks occurred in Sect. 4.1 and analyse root causes of these vulnerabilities in Sect. 4.2. A categorization and summary of countermeasures against vulnerabilities of smart contracts can be seen in Sect. 4.3. Moreover, we present an action-pathway of causes-vulnerabilities-attacks-consequences related to smart contracts in Fig. 4.

4.1 Vulnerabilities and attacks

Smart contracts are commonly used for manipulating data (including digital assets) and enforcement of rules in high-stake financial settings [32], and hence become an attractive target for malicious attackers. Furthermore, smart contracts also become immutable once deployed on the blockchain due to the immutability property of the blockchain. As a result, malicious attackers can exploit security vulnerabilities hidden in the deployed smart contracts to gain profits or sabotage the correct execution of contracts. Among these vulnerabilities, most of them arise from violations of the aforementioned security requirements (Sect. 2.2.1). In this subsection, we analyse several well-known vulnerabilities and some attacks caused by these vulnerabilities.

The reentrancy vulnerability occurs when a smart contract with the fallback function is repeatedly called by another smart contract before the original call is completed (i.e., the smart contract is reentrant) [38, 49, 51–53, 106]. This vulnerability is caused by violating the call integrity of smart contracts and can make the smart contract in an inconsistent state at the point of reentering, resulting in unexpected behaviors [52, 106]. Over the past few years, malicious attackers exploit this vulnerability to launch attacks, causing substantial financial losses. For example, an attacker launched the well-known DAO attack on Ethereum in June 2016 [25] and successfully stole over 3,600,000 Ether (valued at $60 million at the time). Except for the reentrancy vulnerability, Atzei et al. [106] present the "Call to the unknown" vulnerability, which also results from violating the call integrity requirement and causes financial losses.

During the execution of smart contracts, the callee contract should terminate if an exception occurs. However, the propagation policy of exceptions depends on how the call is

made [49, 53, 107]. As a result, there exist many cases where exceptions can not be handled properly. Such mishandled exceptions cause a violation of the atomicity requirement of smart contracts, resulting in several vulnerabilities such as integer overflow, call-stack's depth limit, and unchecked call return values [26, 35, 49]. The BCT-based applications have incurred a number of attacks due to these vulnerabilities. For example, the King of The Ether Throne[13] (KoET) contract has ever been attacked many times by making the current king run out of gas or the call-stack's depth of KoET reach its limit. In such cases, the current king will lose his throne without any compensation (i.e., payments from the attacker). Moreover, recent research works [53] show that 27.9% of contracts do not check return values after calling other contracts, making smart contracts are vulnerable to malicious attacks.

As mentioned in Sect. 2.2.1, the execution of smart contracts should be independent of parameters that are susceptible to the external environment. However, there exist several vulnerabilities that allow attackers to sabotage the execution of smart contracts and gain profits. These vulnerabilities are caused by exploiting the contract's dependencies on environmental variables (e.g., the timestamp contained in the block) or mutable state parameters (e.g., external storage) [49, 51]. Such variables or parameters make smart contracts more prone to a deliberate manipulation by attackers. The block timestamp and the order of transactions to be encapsulated into the new block are mostly utilized, resulting in the well-known timestamp dependency and transactions-ordering dependency [38, 51, 53]. Most of these vulnerabilities are exploited to gain more profits, even manipulate execution outcomes of smart contracts [26].

4.2 Analysis of root causes

Over the past few years, a number of vulnerabilities and attacks related to smart contracts have been discovered. Although there exist a number of research studies on the security of smart contracts, in-depth analysis of the root causes of these vulnerabilities is quite few. In this subsection, we analyse the vulnerabilities and classify the root causes into two categories: unsafe programming before the deployment and unreliable environment during the execution.

Unsafe programming: With growing concerns about smart contracts, numerous programming languages have been applied or proposed to develop smart contracts, including general-purpose languages (e.g., Go, Java, Node.js [108]) and an increasing number of domain-specific languages (e.g., Solidity, Vyper). Domain-specific languages (DSLs) commonly introduce various characteristics to facilitate the development of smart contracts in specific scenarios. For example, DSLs for smart contracts in the financial setting introduce the concept of the gas and call-stack's depth limit to restrict the costly computations incurred during the execution. However, such smart contracts are vulnerable to mishandled exceptions, which result from the introduction of the gas and call-stack's depth limit. Moreover, most programming languages for smart contracts do not provide support for formal semantics and therefore are hard to provide a solid foundation for analysis and verification to detect vulnerabilities. For these reasons, the programming of smart contracts is unsafe and introduces many kinds of vulnerabilities.

What's dominant in unsafe programming is the discrepancy between semantics of the adopted programming language and the intuitive understanding of developers [50, 106, 109]. Vulnerabilities caused by this discrepancy can be exploited to launch attacks, e.g., attacks on the DAO [25] and Parity Wallet [110] contracts. Wherein, malicious attackers exploit the vulnerability caused by the developer's misunderstanding of the semantics of call primitives in a certain programming language to gain more profits [50].

Unreliable execution environment: In the context of the BCT, smart contracts run in an isolated sandbox after the process of development and deployment. The isolated sandbox commonly refers to the Virtual Machine (VM) or Docker, wherein the VM is adopted mostly by various blockchain platforms e.g., Ethereum. As the execution environment for smart contracts, the sandbox's security plays a vital role in the correct execution of smart contracts. In the last two years, multiple previously unknown vulnerabilities were detected in Ethereum Virtual Machine (EVM) [111, 112], which has more than ten widely used implementations supported by different programming languages and several modified versions applied in some open source projects [113]. Such vulnerabilities can be exploited by malicious attackers to make an abnormal call that triggers illegal values or launch more attacks such as a *DoS attack* [111].

There exists another cause that can result in security vulnerabilities during the execution of non-deterministic smart contracts in addition to the two causes mentioned above. Non-deterministic smart contracts refer to a kind of smart contracts that depend on information from external sources (data about the real-world state or events) [114], e.g., a smart contract that requires the latest information about the weather. In such smart contracts, there is no guarantee that the information (data feeds) from external sources is trustworthy. Thus, security vulnerabilities caused by the unauthenticated information lead to a violation of the independence requirement and consequently be exploited by malicious attackers to reap more gains [115].

[13] https://github.com/kieranelby/KingOfTheEtherThrone/blob/v0:4:0/contracts/KingOfTheEtherThrone:sol

Table 3 Programming languages for smart contracts

Categories	Languages	Year	Semantics	Platforms	Active
High-level	Rust	2010	informal	Ethereum	✓
	Solidity	2014	formal	Ethereum	✓
	Mutan	2014	informal	Ethereum	×
	Bamboo	2017	semi	Ethereum	×
	Obsidian	2017	informal	Fabric	✓
	Liquidity	2017	formal	Tezos, Dune Network	✓
	Ivy	2017	informal	Bitcoin	✓
	Vyper	2018	informal	Ethereum	✓
	Flint	2018	informal	Ethereum	✓
	Marlowe	2018	informal	Cardano, Ethereum	✓
	BALZaC	2018	formal	Bitcoin	✓
	BML[a]	2018	formal	Bitcoin	✓
	Plutus	2019	formal	Cardano, IOHK	✓
	LIGO	2019	formal	Tezos	✓
	DAML	2019	formal	Corda	✓
	Move	2019	formal	Libra	✓
	Fe	2020	formal	Ethereum	✓
Intermediate	Yul	2016	informal	Ethereum	✓
	Scilla	2018	formal	Zilliqa	✓
	IELE	2018	formal	Ethereum	✓
	EthIR	2018	informal	Ethereum	✓
	Yul+	2020	informal	Ethereum	✓
	Albert	2020	formal	Tezos	✓
Low-level	BSL[b]	2008	informal	Bitcoin	✓
	LLL	2014	informal	Ethereum	×
	Serpent	2014	informal	Ethereum	×
	EVM	2014	informal	Ethereum	✓
	eWASM	2016	informal	Ethereum	✓
	Simplicity	2017	formal	Bitcoin	✓
	Michelson	2018	formal	Tezos	✓
	Fift	2019	informal	TON	✓
	Huff	2019	informal	Ethereum	✓

[a]: Bitcoin Modelling Language, [b]: Bitcoin Script Language

4.3 Countermeasures

Given the significance of smart contracts in expanding application scenarios of the BCT, there exists a great demand for various techniques to guarantee the security of smart contracts. Over the past few years, extensive research works have been conducted to provide strong support for the correct development and execution of smart contracts and to protect smart contracts deployed on the blockchain from various malicious attacks. To date, most efforts mainly focus on the introduction of safe programming languages and the construction of various analysis tools.

4.3.1 Safe programming languages

As previously mentioned, one of the root causes that result in vulnerabilities of smart contracts is unsafe programming. Safe programming languages is therefore considered as an approach to eliminate potential risks at the design stage. Several efforts have been invested into developing smart contracts with traditional high-level languages (e.g., Go, Java, C++) due to their advantages on developers' familiarity and existing tools' support for verification. Moreover, the approach of safe programming languages gives rise to a rising number of DSLs for developing smart contracts,

including high-level languages for developing more secure smart contracts and intermediate-level languages for facilitating the formal analysis of smart contracts. A summary of these languages used for the development of smart contracts is provided in Table 3.

As shown in Table 3, the majority of programming languages for smart contracts are high-level languages. These programming languages are designed for eliminating vulnerabilities that have been discovered and lead to substantial losses. For example, Flint [116] and Move [117] use linear resource types to protect written smart contracts from vulnerabilities (e.g., the unchecked return value) caused by violating the atomicity requirement. Vyper[14] provides a restricted instruction set (e.g., finite loops and no recursive calls) and introduces new features (e.g., bounds and overflow checking) to eliminate vulnerabilities, which include the DoS with unbounded operations vulnerability, the integer overflow and underflow vulnerability. With the use of polymorphism, Bamboo[15] can mitigate the impact of transactions' orders on the execution outcome of smart contracts and eliminate the reentrancy vulnerability.

Smart contracts written in high-level languages are eventually compiled to code in a low-level language and stored on the blockchain. Wherein low-level languages are used for executing smart contracts in an efficient and deterministic way on the VM or Docker, including EVM for the Ethereum [118] and Michelson[16] for the Tezos. Apart from high-level and low-level languages, there have also been some efforts toward developing intermediate-level languages.

Intermediate-level languages are designed to achieve the expressiveness and tractability of smart contracts and reason about requirements or optimize code. For example, Scilla utilizes a structure based on the communicating automata to provide a clear separation between the communication aspect of contracts, making smart contracts less vulnerable to attacks caused by certain known vulnerabilities and amenable to formal verification [109, 119]. Albert abstracts Michelson (a low-level language for smart contracts in Tezos) stacks through a linear type system to avoid the duplicated effort on the compilation and certification during compiling smart contracts written by high-level languages to Michelson, as well as to extend the application of formal methods for these high-level languages [120]. Intermediate-level languages also include Yul/Yul+[17], EthIR [121], and IELE [122] designed for Ethereum. Compared to the other two categories, intermediate-level languages are deemed as the most suitable languages for formal verification [123] and gradually become one of the emerging directions to develop secure smart contracts.

4.3.2 Techniques and tools

Developing smart contracts with safe programming languages can eliminate some vulnerabilities at the language level. Nevertheless, there is still a need for detecting potential vulnerabilities before deploying smart contracts to the blockchain. As mentioned in Sect. 2.2, smart contracts are essentially programs that are deployed on the blockchain eventually. Therefore, numerous techniques and tools applied to the security analysis of traditional software programs can also be used for analyzing smart contracts, such as symbolic execution, formal verification, and fuzzing. In addition, machine learning (ML), deep learning (DL), and game theory have become emerging approaches to the security analysis of smart contracts as well. Over the past few years, extensive research works on such techniques' applications to smart contracts have been conducted, giving rise to numerous tools or frameworks.

Symbolic execution: Symbolic execution is one of the most widely used techniques to detect vulnerabilities in programs by working on the control-flow graph (CFG) reconstructed from the bytecode of programs [124]. Smart contracts can be executed with symbolic values such that this approach can traverse multiple feasible execution paths on CFG and identify vulnerabilities. Tools employed the symbolic execution technique for the security analysis of smart contracts include Oyente, Maian, VerX, etc. Oyente is one of the early tools proposed for detecting vulnerabilities that violate pre-defined security requirements of smart contracts, such as reentrancy, timestamp dependency, transactions-ordering dependency [53]. To improve the detection capability, Maian [57] and SmartScopy [125] provide support for the consideration of multiple invocations, vulnerable patterns that also violate some security requirements mentioned above, respectively. For mitigating the path explosion problem facing the symbolic execution technique, Maian restricts the invocation path (i.e., the length of a considered sequence of transactions) [57], VerX models the gas mechanics of Ethereum to reduce the paths that are unreachable because of the out-of-gas exception [126], sCompile eliminates the number of suspicious paths that are unrelated to money-transfer [127]. In addition to the code itself to be executed, Manticore presents a symbolic execution engine for VM to detect vulnerabilities in the execution environment of smart contracts [128].

Formal verification: Given the immutability and vulnerabilities that can cause substantial losses, smart contracts are sorely in need of a thorough security analysis before the deployment. Formal verification is an ideal technique that can be used to analyse the security of smart contracts

[14] https://vyper.readthedocs.io/en/latest/?badge=latest

[15] https://github.com/pirapira/bamboo

[16] https://tezos.gitlab.io/whitedoc/michelson.html

[17] https://solidity.readthedocs.io/en/v0.5.10/yul.html

by checking mathematical models against specifications, which specify a set of requirements describing the desired behaviors of smart contracts [38]. The application of formal verification to smart contracts mainly includes model checking, theorem proving, program verification, and runtime verification.

- Model checking performs verification of contract-level models of smart contracts against a temporal logic specification. Such contract-level models focus on high-level behaviors of smart contracts without the consideration of their concrete implementation and execution [129–131]. Similar to symbolic execution, model checking suffers from the state explosion problem [129]. Recently, some research works apply abstraction techniques to smart contracts or simplify the execution of smart contracts to mitigate the impact of the state explosion problem [38, 132].
- Theorem proving performs the verification through encoding smart contracts and their requirements into a particular mathematical logic and deriving a formal proof of satisfaction of these requirements [38]. Currently, efforts on applying the technique of theorem proving to smart contracts involve the development of formal semantics of programming languages and verification frameworks based on some tools such as Coq, Isabelle/HOL [123, 133, 134]. A number of research studies present that theorem proving is greatly conducive to describe and prove the correctness of smart contracts' execution. However, they seldom consider interactions and temporal properties of smart contracts [109, 135, 136] as well as requirements of human involvement and expertise.
- Program verification is used to verify smart contracts by transforming the source code using existing verification languages (e.g., F* [59], WhyML [137], Boogie [138]) and frameworks (e.g., KeY [139] and $\mathbb{K}$ [140]). Based on this technique, several tools have been proposed to automatically detect composite vulnerabilities (i.e., vulnerabilities that can only be exploited through a sequence of transactions [141]) and semantic correctness [52, 58, 138, 142]. Moreover, program verification does not allow verification of liveness requirement, which can be supported by model checking [38, 129].
- Runtime verification is a kind of verification technique that can check security requirements of smart contracts at runtime [26, 38]. Tools based on this technique aim to detect vulnerabilities that violate security requirements during the execution of smart contracts and react to vulnerabilities to mitigate the damage promptly. For example, ContractLarva checks runtime behaviors of smart contracts against behaviors of a new contract generated from the original contract and its specification and takes countermeasures in case of discrepancy [143]. ÆGIS and SODA support verification of an inconsistency between smart contracts and corresponding standards [144, 145]. In terms of the detection capability, runtime verification can identify some vulnerabilities that the aforementioned techniques could not detect, due to their state/path explosion problems or limitation on modeling the complex execution environment.

Fuzzing: Fuzzing (i.e., fuzz testing) is an automatic testing technique that exploits numerous unexpected or invalid inputs to detect vulnerabilities in programs [28, 111]. Currently, several tools based on this technique have been proposed to dynamically analyse the security of smart contracts, e.g., EVMFuzz [111], ContractFuzzer [146], Echidna[18], ReGuard [147], sFuzz [148]. These tools are shown to have a good ability in detecting common vulnerabilities, such as the reentrancy vulnerability. However, most of these tools focus on smart contracts in Ethereum, and part of them are still under development. In order to extend the path coverage of the fuzzer, Viglianisi et al. [149] and Zhang et al. [150] combine fuzzing with symbolic execution and taint analysis.

ML/DL and Game Theory: Apart from the techniques mentioned above, there exist emerging approaches in analyzing the security of smart contracts, such as ML/DL and game theory. ML/DL is used to enhance the efficiency of detecting vulnerabilities in smart contracts [151–153]. For example, ContractWard using the technique of ML learns the vulnerable patterns of smart contracts in training samples and thus detect vulnerabilities by exploiting the parameters learned in the training process without the need for executing the same contract bytecodes repeatedly [153]. Nevertheless, the accuracy and capability of such an approach depend on the correctness of the labelling information and existing discovered vulnerabilities. Recently, some research works exploit game theory to model behaviours of players related to the execution of smart contracts under the assumption of perfect rationality and profit maximization, where dishonest behaviors may occur in game-like interactions [38, 154]. Results from the game-theoretic analysis can be further used for shaping actual behaviors in formal models for automated validation [155].

The aforementioned techniques and tools mainly aim at the vulnerabilities in smart contracts or the execution environment. As we analysed in Sect. 4.2, the security of smart contracts execution depends on the data feeds provided by external sources as well. Therefore, Zhang et al. [115] presents the Town Crier (TC) to provide authenticated data feeds for contracts through acting as a trusted third party between external sources and smart contracts. ChainLink[19]

[18] https://github.com/crytic/echidna/

[19] https://chain.link/

and Adler et al. [156] extend TC by implementing a decentralized oracle network and a voting-based decentralized oracle, respectively, to address the centralized-point-of-failure problem facing TC. Unfortunately, these solutions do not address this problem efficiently as they demonstrate.

5 Discussion

In this section, we discuss key findings based on security analysis for consensus (Sect. 3) and smart contracts (Sect. 4), and then propose several potential research directions.

5.1 Key findings

In previous sections, we have analysed security issues in consensus and smart contracts by an action-pathway of causes-vulnerabilities-attacks-consequences. Furthermore, we have discussed existing countermeasures against security issues in consensus and smart contracts. Based on the analysis in previous sections, we would like to make the following remarks:

1) *Further investigations are still required to make a more comprehensive research on the consensus and smart contract security, while the research should not be restricted to some specific blockchain platform, and in the meanwhile should cover both core blockchain parts (e.g., smart contract programming and programs, consensus algorithms and protocols) and the underlying techniques (e.g., P2P networks)*

Comprehensive and in-depth analysis on blockchain security needs to be conducted, aiming at figuring out which security requirements can be provided by the security techniques, and what kind of actions might violate these requirements and consequently result in vulnerabilities and attacks. Moreover, the analysis should take a broader view instead of limiting the scope to a specific blockchain platform or ignoring the underlying techniques. After all, a general solution is more required to address security challenges facing the BCT, instead of one ad-hoc solution against each security attack.

2) *No silver-bullet that one single measure could provide a perfect countermeasure against most vulnerabilities within consensus or smart contracts at present. Moreover, a countermeasure may solve a security issue partly or work well in some layer of consensus or smart contracts, but it could be of dissatisfaction in many other situations. That highlights the demand for a systematic design of countermeasures.*

There exist few effective countermeasures against most existing vulnerabilities to date, such as the vulnerability of more profitable strategies in consensus protocols, the vulnerability of interference with the routing in P2P networks, and the reentrancy vulnerability in smart contracts. Such vulnerabilities result in a great threat to the security of the BCT. More efforts are required to invest in countering against vulnerabilities in multiple layers of consensus or smart contracts, such as detecting the selfish mining attack or developing smart contracts with safe programming languages. Furthermore, research on the layered countermeasure mechanism is significant in enhancing the security of the BCT.

5.2 Future directions

Based on the security analysis of consensus and smart contracts in previous sections, we would like to outline the following future research directions, aiming to provide a valuable reference for future studies in this area.

1) *Secure consensus*

Research on enhancing the security of consensus can be carried out in the following aspects.

Network-layer security: Research work on the security of P2P networks is much less as compared with that on the security of consensus protocols or smart contracts themselves. Moreover, we learn from the collected data that vulnerabilities at the layer of P2P networks have few effective countermeasures. Even so, P2P networks should be taken into full consideration, because they play an important role in node connection and data transmission that the consensus protocol depends strongly on. Vulnerabilities in P2P networks, the underlying layer of consensus, may influence whether a consensus can be reached [35]. Therefore, a potential research direction for securing consensus lies in studying and enhancing the security of P2P networks.

Effective vulnerability detection and countermeasures design: Existing research on security consensus protocol analysis is mainly based on some impractical assumptions, such as the same communication capability between different nodes and the incentive compatibility of the consensus process. Accordingly, it is hard to perform effective detection and countermeasures design against vulnerabilities (e.g., selfish mining) in consensus protocols. Therefore, research on vulnerabilities detection and effective countermeasures is also a promising direction.

Joint optimization on consensus protocols and incentive mechanisms: Single optimization on consensus protocols or incentive mechanisms has poor performance in countering against vulnerabilities at the consensus layer. Thus, a joint optimization is regarded as a promising direction to address the existing and more potential vulnerabilities at this layer.

2) *Secure smart contracts*

Based on the security analysis of smart contracts in previous sections, we envision the following research directions in the area of securing smart contracts.

Designing and developing by security: Developing smart contracts with well design and safe programming languages is one of the emerging research directions in this area, aiming at eliminating many risks at the early stage of development. For example, an increasing number of domain-specific languages have been proposed to simplify the development and verification of smart contracts, e.g., Findel [157] and Marlowe [158].

Fault-tolerant smart contracts: Developing fault-tolerant smart contracts is also a promising research direction that deserves more attention. This kind of smart contracts are expected to stop the execution and recover from runtime errors (a.k.a, automated repair of smart contracts). To date, there exist some efforts invested in this direction, e.g., Yu et al. [159] exploit an automated gas-aware repair technique for smart contracts in Ethereum.

Analysis using a combination of techniques: A combination of multiple techniques might achieve better results than a single technique when analysing smart contracts' security. For example, He et al. [160] combine fuzzing and symbolic execution to navigate a fuzzer towards a higher path coverage, Chatterjee et al. [154] and Liu et al. [161] analyse the execution of smart contracts in the context of multi-participants and complex interactions through the combination of game theory and formal verification.

3) *A comprehensive security analysis framework*

Extensive research works have been done to analyse the security of consensus or smart contracts or both. Few of them provide a comprehensive discussion on security requirements and evaluation. Hence, there is an urgent need to grasp the desirable security requirements of the BCT, to design a principled and rigorous methodology to analyse whether these requirements can be satisfied in a practical setting, to exploit existing security metrics or develop new metrics to quantify the security of the BCT. Briefly, there is an urgent need for a comprehensive security analysis framework.

6 Conclusion

BCT has received lots of extensive attention over the past decade owing to its characteristics such as transparency, tamper resistance, traceability, etc. As introducing smart contracts, the new *Blockchain 2.0* has been applied to rich application scenarios ranging from cryptocurrency to IoT, healthcare, electronic voting, supply chain, etc., and meanwhile importing a lot of vulnerabilities and attacks within consensus and smart contracts. In the light of security requirements for extended applications, it's essential to perform a comprehensive and systematic analysis of the BCT security issues.

We mainly discuss the security of consensus and smart contracts on general blockchain platforms from an integral perspective that includes four steps in an action-pathway from root causes, vulnerabilities, and attacks, to the consequences. Moreover, the proposed countermeasures to the consensus and smart contract security issues are also categorized, evaluated, and discussed in a holistic manner. Through discussion on the findings, we believe that future research works should design countermeasures with full consideration of the action-pathway of causes-vulnerabilities-attacks-consequences. We expect the current work could provide a comprehensive understanding of BCT's security issues and enlighten systematic thinking for the future design of secure blockchain.

We also admit that this survey paper is far from a complete view that contains all security requirements and explains thoroughly the existing or underlying security issues related to consensus or smart contracts. The current work could be further improved by including and analysing more security vulnerabilities and attacks.

Author contributions Bo Liu designed the research. Xuelian Cao, Jianhui Zhang, and Xuechen Wu performed the literature search and data analysis. Xuelian Cao and Bo Liu drafted the manuscript. Jianhui Zhang and Xuechen Wu helped organise the manuscript. Xuelian Cao and Bo Liu revised and finalized the paper.

Funding information Project is supported in part by the Special Foundation for Basic Science and Frontier Technology Research Program of Chongqing (No. cstc2017jcyjAX0295), the Capacity Development Foundation of Southwest University (No. SWU116007), and the National Natural Science Foundation of China (No.61732019, 62032019, 61872051).

Declarations

Conflicts of interest Xuelian Cao, Jianhui Zhang, Xuechen Wu, and Bo Liu declare that they have no conflict of interest.

References

1. Kogure J, Kamakura K, Shima T (2017) Blockchain Technology for Next Generation ICT. Fujitsu Sci Tech J 53(5):56–61
2. Kagan J (2020) Financial Technology Fintech. https://www.investopedia.com/terms/f/fintech.asp. Accessed 29 Nov 2020
3. Berg C, Davidson S, Potts J (2019) Blockchain Technology as Economic Infrastructure: Revisiting the Electronic Markets Hypothesis. Frontiers in Blockchain 2:22

4. Ko T, Lee J, Ryu D (2018) Blockchain Technology and Manufacturing Industry: Real-Time Transparency and Cost Savings. Sustainability 10(11):4274
5. Nakamoto S (2008) Bitcoin : A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf. Accessed 29 Nov 2020
6. Yaga D, Mell P, Roby N, Scarfone K (2018) Blockchain technology overview. https://nvlpubs.nist.gov/nistpubs/ir/2018/NIST.IR.8202.pdf. Accessed 29 Nov 2020
7. Das P, Eckey L, Frassetto T, Gens D, Hostáková K, Jauernig P, Faust S, Sadeghi A (2019) FastKitten: Practical Smart Contracts on Bitcoin. In: 28th USENIX Security Symposium, USENIX Association, pp 801–818
8. Szabo N (1996) Smart Contracts : Building Blocks for Digital Markets. https://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwinterschool2006/szabo.best.vwh.net/smart_contracts_2.html. Accessed 29 Nov 2020
9. Zhu Y, Zhang X, Ju ZY, Wang C (2020) A study of blockchain technology development and military application prospects. J Phys: Conf Ser 1507
10. Buterin V (2013) A Next-Generation Smart Contract and Decentralized Application Platform. https://ethereum.org/en/whitepaper/. Accessed 29 Nov 2020
11. Johnson M, Jones M, Shervey M, Dudley JT, Zimmerman N (2019) Building a Secure Biomedical Data Sharing Decentralized App (DApp): Tutorial 21(10):e13601
12. Davidson S, De Filippi P, Potts J (2016) Economics of Blockchain. http://www.ssrn.com/abstract=2744751. Accessed 29 Nov 2020
13. Ali MS, Vecchio M, Pincheira M, Dolui K, Antonelli F, Rehmani MH (2019) Applications of Blockchains in the Internet of Things: A Comprehensive Survey 21(2):1676–1717
14. Tan L, Shi N, Yu K, Aloqaily M, Jararweh Y (2021a) A Blockchain-empowered Access Control Framework for Smart Devices in Green Internet of Things. ACM Transactions on Internet Technology 21(3):80:1–80:20
15. Yu K, Tan L, Aloqaily M, Yang H, Jararweh Y (2021) Blockchain-enhanced data sharing with traceable and direct revocation in iiot. IEEE Trans Industr Inf 17(11):7669–7678
16. Schar F (2020) Decentralized Finance: On Blockchain- and Smart Contract-based Financial Markets. https://papers.ssrn.com/abstract=3571335. Accessed 29 Nov 2020
17. Kundu D (2019) Blockchain and Trust in a Smart City. Environ Urban ASIA 10(1):31–43
18. Singh P, Nayyar A, Kaur A, Ghosh U (2020) Blockchain and fog based architecture for internet of everything in smart cities. Future Internet 12(4):61
19. Tan L, Xiao H, Yu K, Aloqaily M, Jararweh Y (2021b) A blockchain-empowered crowdsourcing system for 5g-enabled smart cities. Computer Standards & Interfaces 76:103517
20. Viriyasitavat W, Xu LD, Bi Z, Pungpapong V (2019) Blockchain and Internet of Things for Modern Business Process in Digital Economy the State of the Art. IEEE Trans Comput Soc Syst 6(6):1420–1432
21. Frikha T, Chaabane F, Aouinti N, Cheikhrouhou O, Ben Amor N, Kerrouche A (2021) Implementation of Blockchain Consensus Algorithm on Embedded Architecture. Security and Communication Networks 2021
22. Tayal A, Solanki A, Kondal R, Nayyar A, Tanwar S, Kumar N (2021) Blockchain-based efficient communication for food supply chain industry: Transparency and traceability analysis for sustainable business. Int J Commun Syst 34(4)
23. Jiang Z, Cao Z, Krishnamachari B, Zhou S, Niu Z (2020) SENATE: A Permissionless Byzantine Consensus Protocol in Wireless Networks for Real-Time Internet-of-Things Applications. IEEE Internet Things J 7(7):6576–6588
24. McAfee (2018) Blockchain Threat Report. https://www.mcafee.com/enterprise/en-us/assets/reports/rp-blockchain-security-risks.pdf. Accessed 30 Nov 2020
25. Daian P (2016) Analysis of the DAO exploit. https://hackingdistributed.com/2016/06/18/analysis-of-the-dao-exploit/. Accessed 29 Nov 2020
26. Chen H, Pendleton M, Njilla L, Xu S (2020a) A Survey on Ethereum Systems Security: Vulnerabilities, Attacks, and Defenses. ACM Computing Surveys 53(3):67:1–67:43
27. Cheng J, Xie L, Tang X, Xiong N, Liu B (2020) A survey of security threats and defense on Blockchain. In: Multimedia Tools and Applications, Springer
28. Homoliak I, Venugopalan S, Reijsbergen D, Hum Q, Schumi R, Szalachowski P (2021) The Security Reference Architecture for Blockchains: Towards a Standardized Model for Studying Vulnerabilities, Threats, and Defenses. IEEE Communications Surveys & Tutorials 23(1):341–390
29. Samreen NF, Alalfi MH (2021) A Survey of Security Vulnerabilities in Ethereum Smart Contracts. CoRR abs/2105.06974
30. Zaghloul E, Li T, Mutka M, Ren J (2020) Bitcoin and Blockchain: Security and Privacy. IEEE Internet Things J 7(10):10288–10313
31. Kolb J, AbdelBaky M, Katz RH, Culler DE (2020) Core Concepts, Challenges, and Future Directions in Blockchain: A Centralized Tutorial. ACM Computing Surveys 53(1):9:1–9:39
32. Wang Z, Jin H, Dai W, Choo KR, Zou D (2021) Ethereum smart contract security research: survey and future research opportunities. Front Comp Sci 15(2)
33. Dasgupta D, Shrein JM, Gupta KD (2019) A survey of blockchain from security perspective. J Bank Financial Tech 3(1):1–17
34. Leng J, Zhou M, Zhao JL, Huang Y, Bian Y (2021) Blockchain Security: A Survey of Techniques and Research Directions. IEEE Trans Serv Comput 51(1):237–252
35. Saad M, Spaulding J, Njilla L, Kamhoua CA, Shetty S, Nyang D, Mohaisen A (2020) Exploring the Attack Surface of Blockchain: A Comprehensive Survey. IEEE Communications Surveys & Tutorials 22(3):1977–2008
36. Zhang R, Xue R, Liu L (2019) Security and Privacy on Blockchain. ACM Computing Surveys 52(3):51:1–51:34
37. Kim S, Ryu S (2020) Analysis of Blockchain Smart Contracts: Techniques and Insights. In: IEEE Secure Development (SecDev), IEEE, pp 65–73
38. Tolmach P, Li Y, Lin S, Liu Y, Li Z (2021) A Survey of Smart Contract Formal Specification and Verification. ACM Computing Surveys 54(7):141:1–141:38
39. Dotan M, Pignolet YA, Schmid S, Tochner S, Zohar A (2021) Survey on Blockchain Networking: Context, State-of-the-Art, Challenges. ACM Computing Surveys 54(5):107:1–107:34
40. Li D, Deng L, Gupta BB, Wang H, Choi C (2019a) A novel CNN based security guaranteed image watermarking generation scenario for smart city applications. Information Sciences 479:432–447
41. Schollmeier R (2001) A Definition of Peer-to-Peer Networking for the Classification of Peer-to-Peer Architectures and Applications. In: 1st International Conference on Peer-to-Peer Computing (P2P), IEEE Computer Society, pp 101–102
42. Donet Donet JA, Pérez-Solà C, Herrera-Joancomartí J (2014) The Bitcoin P2P Network. In: Financial Cryptography and Data Security (FC), Springer, Lecture Notes in Computer Science, vol 8438, pp 87–102
43. Jain S, Mahajan R, Wetherall D (2003) A Study of the Performance Potential of DHT-based Overlays. In: 4th USENIX Symposium on Internet Technologies and Systems (USITS), USENIX Association
44. Lamport L, Shostak R, Pease M (1982) The Byzantine Generals Problem. ACM Trans Program Lang Syst 4(3):382–401
45. Satoshi N (2008) Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf. Accessed 29 Nov 2020

46. Castro M, Liskov B (2002) Practical byzantine fault tolerance and proactive recovery. ACM Trans Comp Syst 20(4):398–461
47. Bano S, Sonnino A, Al-Bassam M, Azouvi S, McCorry P, Meiklejohn S, Danezis G (2019) SoK: Consensus in the Age of Blockchains. In: Proceedings of the 1st ACM Conference on Advances in Financial Technologies (AFT), ACM, pp 183–198
48. Szabo N (1997) Formalizing and Securing Relationships on Public Networks. First Monday 2(9)
49. Grishchenko I, Maffei M, Schneidewind C (2018a) A Semantic Framework for the Security Analysis of Ethereum Smart Contracts. In: Principles of Security and Trust (POST), Springer, Lecture Notes in Computer Science, vol 10804, pp 243–269
50. Grishchenko I, Maffei M, Schneidewind C (2018b) Foundations and Tools for the Static Analysis of Ethereum Smart Contracts. In: International Conference on Computer Aided Verification (CAV), Springer, Lecture Notes in Computer Science, vol 10981, pp 51–78
51. Harz D, Knottenbelt WJ (2018) Towards Safer Smart Contracts: A Survey of Languages and Verification Methods. CoRR abs/1809.09805
52. Schneidewind C, Grishchenko I, Scherer M, Maffei M (2020) eThor: Practical and Provably Sound Static Analysis of Ethereum Smart Contracts. In: ACM SIGSAC Conference on Computer and Communications Security (CCS), ACM, pp 621–640
53. Luu L, Chu DH, Olickel H, Saxena P, Hobor A (2016) Making Smart Contracts Smarter. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), ACM, pp 254–269
54. Zupan N, Kasinathan P, Cuellar J, Sauer M (2020) Secure Smart Contract Generation Based on Petri Nets. In: Blockchain Technology for Industry 4.0: Secure, Decentralized, Distributed and Trusted Industry Environment, Springer, pp 73–98
55. Wang S, Zhang C, Su Z (2019a) Detecting nondeterministic payment bugs in Ethereum smart contracts. Proceedings of the ACM on Programming Languages 3(OOPSLA):189:1–189:29
56. Bartoletti M, Zunino R (2019) Verifying Liquidity of Bitcoin Contracts. In: Principles of Security and Trust (POST), Springer, Lecture Notes in Computer Science, vol 11426, pp 222–247
57. Nikolic I, Kolluri A, Sergey I, Saxena P, Hobor A (2018) Finding The Greedy, Prodigal, and Suicidal Contracts at Scale. In: Proceedings of the 34th Annual Computer Security Applications Conference (ACSAC), ACM, pp 653–663
58. Tsankov P, Dan AM, Drachsler-Cohen D, Gervais A, Bünzli F, Vechev MT (2018) Securify: Practical Security Analysis of Smart Contracts. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS), ACM, pp 67–82
59. Bhargavan K, Delignat-Lavaud A, Fournet C, Gollamudi A, Gonthier G, Kobeissi N, Kulatova N, Rastogi A, Sibut-Pinote T, Swamy N, Béguelin SZ (2016) Formal Verification of Smart Contracts: Short Paper. In: Proceedings of the 2016 ACM Workshop on Programming Languages and Analysis for Security, ACM, pp 91–96
60. Chen T, Li X, Luo X, Zhang X (2017) Under-optimized smart contracts devour your money. 24th International Conference on Software Analysis. IEEE Computer Society, Evolution and Reengineering (SANER), pp 442–446
61. Grech N, Kong M, Jurisevic A, Brent L, Scholz B, Smaragdakis Y (2018) MadMax: surviving out-of-gas conditions in Ethereum smart contracts. Proceedings of the ACM on Programming Languages 2(OOPSLA):116:1–116:27
62. Douceur JR (2002) The Sybil Attack. Peer-to-Peer Systems, Springer, Lecture Notes in Computer Science 2429:251–260
63. Carrara G, Burle L, Medeiros D, Albuquerque C, Menezes D (2020) Consistency, availability, and partition tolerance in blockchain: a survey on the consensus mechanism over peer-to-peer networking. Ann Telecommun 75:163–174
64. Ekparinya P, Gramoli V, Jourjon G (2020) The Attack of the Clones Against Proof-of-Authority. In: 27th Annual Network and Distributed System Security Symposium (NDSS), The Internet Society
65. Heilman E, Kendler A, Zohar A, Goldberg S (2015) Eclipse Attacks on Bitcoin's Peer-to-Peer Network. In: Proceedings of the 24th USENIX Conference on Security Symposium, USENIX Association, pp 129–144
66. Wiki B (2018) Confirmation. https://en.bitcoin.it/wiki/Confirmation. Accessed 29 Nov 2020
67. Eyal I, Sirer EG (2014) Majority Is Not Enough: Bitcoin Mining Is Vulnerable. In: Financial Cryptography and Data Security (FC), Springer, Lecture Notes in Computer Science, vol 8437, pp 436–454
68. Apostolaki M, Zohar A, Vanbever L (2017) Hijacking Bitcoin: Routing Attacks on Cryptocurrencies. In: IEEE Symposium on Security and Privacy (SP), IEEE Computer Society, pp 375–392
69. Ekparinya P, Gramoli V, Jourjon G (2018) Impact of Man-In-The-Middle Attacks on Ethereum. In: 37th IEEE Symposium on Reliable Distributed Systems (SRDS), IEEE Computer Society, pp 11–20
70. Natoli C, Gramoli V (2017) The Balance Attack or Why Forkable Blockchains are Ill-Suited for Consortium. In: 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), IEEE Computer Society, pp 579–590
71. Baumann A, Fabian B, Lischke M (2014) Exploring the Bitcoin Network. In: Proceedings of the 10th International Conference on Web Information Systems and Technologies (WEBIST), SciTePress, vol 1, pp 369–374
72. Houy N (2016) The Bitcoin Mining Game. Ledger 1:53–68
73. Xiao Y, Zhang N, Lou W, Hou YT (2020) Modeling the Impact of Network Connectivity on Consensus Security of Proof-of-Work Blockchain. In: 39th IEEE Conference on Computer Communications (INFOCOM), IEEE, pp 1648–1657
74. Xiong Z, Feng S, Niyato D, Wang P, Han Z (2018) Optimal Pricing-Based Edge Computing Resource Management in Mobile Blockchain. In: IEEE International Conference on Communications (ICC), IEEE, pp 1–6
75. Eyal I (2015) The Miner's Dilemma. In: IEEE Symposium on Security and Privacy (SP), IEEE Computer Society, pp 89–103
76. Draupnir M (2016) Bitcoin Mining Centralization. https://www.bitcoinmining.com/bitcoin-mining-centr-alization/. Accessed 29 Nov 2020
77. Sapirshtein A, Sompolinsky Y, Zohar A (2016) Optimal Selfish Mining Strategies in Bitcoin. In: Financial Cryptography and Data Security (FC), Springer, Lecture Notes in Computer Science, vol 9603, pp 515–532
78. Szalachowski P, Reijsbergen D, Homoliak I, Sun S (2019) StrongChain: Transparent and Collaborative Proof-of-Work Consensus. In: 28th USENIX Security Symposium, USENIX Association, pp 819–836
79. Tsabary I, Eyal I (2018) The Gap Game. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS), ACM, pp 713–728
80. Nayak K, Kumar S, Miller A, Shi E (2016) Stubborn Mining: Generalizing Selfish Mining and Combining with an Eclipse Attack. In: IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, pp 305–320
81. Dong X, Wu F, Faree A, Guo D, Shen Y, Ma J (2019) Selfholding: A combined attack model using selfish mining with block withholding attack. Computer & Security 87
82. Kwon Y, Kim D, Son Y, Vasserman EY, Kim Y (2017) Be Selfish and Avoid Dilemmas: Fork After Withholding (FAW) Attacks on Bitcoin. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS), ACM, pp 195–209

83. Sompolinsky Y, Zohar A (2016) Bitcoin's Security Model Revisited. CoRR abs/1605.09193
84. Churyumov A (2016) Byteball: A decentralized system for storage and transfer of value. https://byteball.org/Byteball.pdf. Accessed 29 Nov 2020
85. COTI (2018) COTI: a decentralized, high performance cryptocurrency ecosystem optimized for creating digital payment networks and stable coins. https://coti.io/files/COTI-technical-whitepaper.pdf. Accessed 29 Nov 2020
86. Garay JA, Kiayias A, Leonardos N (2015) The Bitcoin Backbone Protocol: Analysis and Applications. In: Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part II, Springer, Lecture Notes in Computer Science, vol 9057, pp 281–310
87. Negy KA, Rizun PR, Sirer EG (2020) Selfish Mining Re-Examined. In: Financial Cryptography and Data Security (FC), Springer, Lecture Notes in Computer Science, vol 12059, pp 61–78
88. Zhang R, Preneel B (2019) Lay Down the Common Metrics: Evaluating Proof-of-Work Consensus Protocols' Security. In: IEEE Symposium on Security and Privacy (S&P), IEEE, pp 175–192
89. Tran M, Choi I, Moon GJ, Vu AV, Kang MS (2020) A Stealthier Partitioning Attack against Bitcoin Peer-to-Peer Network. In: IEEE Symposium on Security and Privacy (S&P), IEEE, pp 894–909
90. Alangot B, Reijsbergen D, Venugopalan S, Szalachowski P (2020) Decentralized Lightweight Detection of Eclipse Attacks on Bitcoin Clients. In: IEEE International Conference on Blockchain, IEEE, pp 337–342
91. Apostolaki M, Marti G, Müller J, Vanbever L (2019) SABRE: Protecting Bitcoin against Routing Attacks. In: 26th Annual Network and Distributed System Security Symposium (NDSS), The Internet Society
92. Luu L, Velner Y, Teutsch J, Saxena P (2017) SmartPool: Practical Decentralized Pooled Mining. In: 26th USENIX Security Symposium, USENIX Association, pp 1409–1426
93. Dey S (2018) Securing Majority-Attack in Blockchain Using Machine Learning and Algorithmic Game Theory: A Proof of Work. In: 10th Computer Science and Electronic Engineering Conference (CEEC), IEEE, pp 7–10
94. Wang Y, Tang C, Lin F, Zheng Z, Chen Z (2019b) Pool Strategies Selection in PoW-Based Blockchain Networks: Game-Theoretic Analysis. IEEE Access 7:8427–8436
95. Chicarino VRL, Albuquerque C, Jesus EF, de A Rocha AA (2020) On the detection of selfish mining and stalker attacks in blockchain networks. Annals of Telecommunications 75(3–4), 143–152
96. Hou C, Zhou M, Ji Y, Daian P, Tramèr F, Fanti G, Juels A (2021) SquirRL: Automating Attack Analysis on Blockchain Incentive Mechanisms with Deep Reinforcement Learning. In: 28th Annual Network and Distributed System Security Symposium (NDSS), The Internet Society
97. Saad M, Njilla L, Kamhoua CA, Mohaisen A (2019) Countering Selfish Mining in Blockchains. International Conference on Computing. Networking and Communications (ICNC), IEEE, pp 360–364
98. Buchman E, Kwon J, Milosevic Z (2018) The latest gossip on BFT consensus. CoRR abs/1807.04938
99. Kokoris-Kogias E, Jovanovic P, Gailly N, Khoffi I, Gasser L, Ford B (2016) Enhancing Bitcoin Security and Performance with Strong Consistency via Collective Signing. In: 25th USENIX Security Symposium, USENIX Association, pp 279–296
100. Lerner SD (2015) DECOR+HOP: A Scalable Blockchain Protocol. https://scalingbitcoin.org/papers/DECOR-HOP.pdf. Accessed 29 Nov 2020
101. Eyal I, Sirer EG (2018) Majority is not enough: bitcoin mining is vulnerable. Commun ACM 61(7):95–102
102. Pass R, Shi E (2017) FruitChains: A Fair Blockchain. In: Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC), ACM, pp 315–324
103. Bissias G, Levine BN (2020) Bobtail: Improved Blockchain Security with Low-Variance Mining. In: 27th Annual Network and Distributed System Security Symposium (NDSS), The Internet Society
104. Camacho P, Lerner SD (2016) DECOR+LAMI: A Scalable Blockchain Protocol. https://scalingbitcoin.org/papers/DECOR-LAMI.pdf. Accessed 29 Nov 2020
105. Zhang R, Preneel B (2017) Publish or Perish: A Backward-Compatible Defense Against Selfish Mining in Bitcoin. In: Handschuh H (ed) Cryptographers' Track at the RSA Conference (CT-RSA), Springer, Lecture Notes in Computer Science, vol 10159, pp 277–292
106. Atzei N, Bartoletti M, Cimoli T (2017) A Survey of Attacks on Ethereum Smart Contracts (SoK). Principles of Security and Trust, Springer, Lecture Notes in Computer Science 10204:164–186
107. Pérez D, Livshits B (2019) Smart Contract Vulnerabilities: Does Anyone Care? CoRR abs/1902.06710
108. Cachin C (2016) Architecture of the Hyperledger Blockchain Fabric. https://www.zurich.ibm.com/dccl/papers/cachin_dcc-l.pdf. Accessed 29 Nov 2020
109. Sergey I, Nagaraj V, Johannsen J, Kumar A, Trunov A, Hao KCG (2019) Safer smart contract programming with Scilla. Proceedings of the ACM on Programming Languages 3(OOPSLA):185:1–185:30
110. Alois J (2017) Ethereum Parity Hack May Impact ETH 500,000 or $146 Million. https://www.crowdfundinsider.com/2017/11/124200-ethereum-parity-hack-may-impact-eth-500000-146-million/. Accessed 29 Nov 2020
111. Fu Y, Ren M, Ma F, Shi H, Yang X, Jiang Y, Li H, Shi X (2019) EVMFuzzer: detect EVM vulnerabilities via fuzz testing. In: Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), ACM, pp 1110–1114
112. Sotnichek M (2018) Blockchain vulnerabilities: Fomo3D exploit explained. https://www.apriorit.com/dev-blog/556-fomo3d-vulne-rability. Accessed 29 Nov 2020
113. Ethereum (2018) Ethereum Virtual Machine (EVM) Implementations. https://eth.wiki/concepts/evm/implementations. Accessed 29 Nov 2020
114. Alharby M, van Moorsel A (2017) Blockchain-based Smart Contracts: A Systematic Mapping Study. CoRR abs/1710.06372
115. Zhang F, Cecchetti E, Croman K, Juels A, Shi E (2016) Town Crier: An Authenticated Data Feed for Smart Contracts. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), ACM, pp 270–282
116. Schrans F, Eisenbach S, Drossopoulou S (2018) Writing safe smart contracts in Flint. In: Conference Companion of the 2nd International Conference on Art, Science, and Engineering of Programming, ACM, pp 218–219
117. Blackshear S, Dill DL, Qadeer S, Barrett CW, Mitchell JC, Padon O, Zohar Y (2020) Resources: A Safe Language Abstraction for Money. CoRR abs/2004.05106
118. Wood G (2014) Ethereum: a secure decentralised generalised transaction ledger. http://gavwood.com/paper.pdf. Accessed 29 Nov 2020
119. Sergey I, Kumar A, Hobor A (2018a) Scilla: a Smart Contract Intermediate-Level LAnguage. CoRR abs/1801.00687
120. Bernardo B, Cauderlier R, Pesin B, Tesson J (2020) Albert, An Intermediate Smart-Contract Language for the Tezos Blockchain. In: Financial Cryptography and Data Security (FC),

Springer, Lecture Notes in Computer Science, vol 12063, pp 584–598
121. Albert E, Gordillo P, Livshits B, Rubio A, Sergey I (2018) EthIR: A Framework for High-Level Analysis of Ethereum Bytecode. In: Automated Technology for Verification and Analysis (ATVA), Springer, Lecture Notes in Computer Science, vol 11138, pp 513–520
122. Kasampalis T, Guth D, Moore BM, Serbanuta T, Zhang Y, Filaretti D, Serbanuta VN, Johnson R, Rosu G (2019) IELE: A Rigorously Designed Language and Tool Ecosystem for the Blockchain. In: International Symposium on Formal Methods (FM), Springer, Lecture Notes in Computer Science, vol 11800, pp 593–610
123. Li X, Shi Z, Zhang Q, Wang G, Guan Y, Han N (2019b) Towards Verifying Ethereum Smart Contracts at Intermediate Language Level. In: 21st International Conference on Formal Engineering Methods (ICFEM), Springer, Lecture Notes in Computer Science, vol 11852, pp 121–137
124. Cadar C, Sen K (2013) Symbolic execution for software testing: three decades later. Commun ACM 56(2):82–90
125. Feng Y, Torlak E, Bodík R (2019) Precise Attack Synthesis for Smart Contracts. CoRR abs/1902.06067
126. Permenev A, Dimitrov D, Tsankov P, Drachsler-Cohen D, Vechev MT (2020) VerX: Safety Verification of Smart Contracts. In: IEEE Symposium on Security and Privacy (S&P), IEEE, pp 1661–1677
127. Chang J, Gao B, Xiao H, Sun J, Cai Y, Yang Z (2019) sCompile: Critical Path Identification and Analysis for Smart Contracts. In: 21st International Conference on Formal Engineering Methods (ICFEM), Springer, Lecture Notes in Computer Science, vol 11852, pp 286–304
128. Mossberg M, Manzano F, Hennenfent E, Groce A, Grieco G, Feist J, Brunson T, Dinaburg A (2019) Manticore: A User-Friendly Symbolic Execution Framework for Binaries and Smart Contracts. In: 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, pp 1186–1189
129. Nehai Z, Piriou P, Daumas FF (2018) Model-Checking of Smart Contracts. IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber. Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData), IEEE, pp 980–987
130. Nelaturu K, Mavridou A, Veneris A, Laszka A (2020) Verified Development and Deployment of Multiple Interacting Smart Contracts with VeriSolid. In: International Conference on Blockchain and Cryptocurrency (ICBC), IEEE, pp 1–9
131. Osterland T, Rose T (2020) Model checking smart contracts for Ethereum. Pervasive Mob Comput 63
132. Kongmanee J, Kijsanayothin P, Hewett R (2019) Securing Smart Contracts in Blockchain. In: 34th IEEE/ACM International Conference on Automated Software Engineering (ASE) Workshops, IEEE, pp 69–76
133. Amani S, Bégel M, Bortin M, Staples M (2018) Towards verifying ethereum smart contract bytecode in Isabelle/HOL. In: Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs, ACM, pp 66–77
134. Bernardo B, Cauderlier R, Hu Z, Pesin B, Tesson J (2019) Mi-Cho-Coq, a Framework for Certifying Tezos Smart Contracts. In: International Symposium on Formal Methods (FM), Springer, Lecture Notes in Computer Science, vol 12232, pp 368–379
135. Nielsen JB, Spitters B (2019) Smart Contract Interactions in Coq. In: International Symposium on Formal Methods (FM), Springer, Lecture Notes in Computer Science, vol 12232, pp 380–391
136. Sergey I, Kumar A, Hobor A (2018b) Temporal Properties of Smart Contracts. In: Leveraging Applications of Formal Methods, Verification and Validation, Springer, Lecture Notes in Computer Science, vol 11247, pp 323–338
137. da Horta LPA, Reis JS, Pereira M, de Sousa SM (2020) WhyLSon: Proving your Michelson Smart Contracts in Why3. CoRR abs/2005.14650
138. Lahiri SK, Chen S, Wang Y, Dillig I (2018) Formal Specification and Verification of Smart Contracts for Azure Blockchain. CoRR abs/1812.08829
139. Ahrendt W, Bubel R, Ellul J, Pace GJ, Pardo R, Rebiscoul V, Schneider G (2019) Verification of Smart Contract Business Logic - Exploiting a Java Source Code Verifier. In: Fundamentals of Software Engineering (FSEN), Springer, Lecture Notes in Computer Science, vol 11761, pp 228–243
140. Park D, Zhang Y, Saxena M, Daian P, Rosu G (2018) A formal verification tool for Ethereum VM bytecode. In: Proceedings of the 2018 ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), ACM, pp 912–915
141. Brent L, Grech N, Lagouvardos S, Scholz B, Smaragdakis Y (2020) Ethainter: a smart contract security analyzer for composite vulnerabilities. In: Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI), ACM, pp 454–469
142. Feist J, Grieco G, Groce A (2019) Slither: a static analysis framework for smart contracts. In: Proceedings of the 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB), IEEE, pp 8–15
143. Ellul J, Pace GJ (2018) Runtime Verification of Ethereum Smart Contracts. In: 14th European Dependable Computing Conference (EDCC), IEEE Computer Society, pp 158–163
144. Chen T, Cao R, Li T, Luo X, Gu G, Zhang Y, Liao Z, Zhu H, Chen G, He Z, Tang Y, Lin X, Zhang X (2020c) SODA: A Generic Online Detection Framework for Smart Contracts. In: 27th Annual Network and Distributed System Security Symposium (NDSS), The Internet Society
145. Torres CF, Baden M, Norvill R, Jonker H (2019) ÆGIS: Smart Shielding of Smart Contracts. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS), ACM, pp 2589–2591
146. Jiang B, Liu Y, Chan WK (2018) ContractFuzzer: fuzzing smart contracts for vulnerability detection. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE), ACM, pp 259–269
147. Liu C, Liu H, Cao Z, Chen Z, Chen B, Roscoe B (2018) ReGuard: finding reentrancy bugs in smart contracts. In: Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings (ICSE), ACM, pp 65–68
148. Nguyen TD, Pham LH, Sun J, Lin Y, Minh QT (2020) sFuzz: An Efficient Adaptive Fuzzer for Solidity Smart Contracts. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE), ACM, p 778-788
149. Viglianisi E, Ceccato M, Tonella P (2020) A federated society of bots for smart contract testing. J Syst Softw 168
150. Zhang Q, Wang Y, Li J, Ma S (2020) EthPloit: From Fuzzing to Efficient Exploit Generation against Smart Contracts. 27th IEEE Int Conf Soft Anal. Evolution and Reengineering (SANER), IEEE, pp 116–126
151. Chen J, Xia X, Lo D, Grundy JC (2020b) Why Do Smart Contracts Self-Destruct? Investigating the Selfdestruct Function on Ethereum. CoRR abs/2005.07908
152. Gao Z, Jayasundara V, Jiang L, Xia X, Lo D, Grundy JC (2019) SmartEmbed: A Tool for Clone and Bug Detection in Smart Contracts through Structural Code Embedding. In: International Conference on Software Maintenance and Evolution (ICSME), IEEE, pp 394–397
153. Wang W, Song J, Xu G, Li Y, Wang H, Su C (2021) ContractWard: Automated Vulnerability Detection Models for Ethereum Smart Contracts. IEEE Trans Netw Sci Eng 8(2):1133–1144

154. Chatterjee K, Goharshady AK, Velner Y (2018) Quantitative Analysis of Smart Contracts. Programming Languages and Systems, Springer, Lecture Notes in Computer Science 10801:739–767
155. Laneve C, Coen CS, Veschetti A (2019) On the Prediction of Smart Contracts' Behaviours. From Software Engineering to Formal Methods and Tools, and Back, Springer, Lecture Notes in Computer Science 11865:397–415
156. Adler J, Berryhill R, Veneris AG, Poulos Z, Veira N, Kastania A (2018) Astraea: A Decentralized Blockchain Oracle. IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber. Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData), IEEE, pp 1145–1152
157. Biryukov A, Khovratovich D, Tikhomirov S (2017) Findel: Secure Derivative Contracts for Ethereum. In: Financial Cryptography and Data Security (FC), Springer, Lecture Notes in Computer Science, vol 10323, pp 453–467
158. Seijas PL, Nemish A, Smith D, Thompson SJ (2020) Marlowe: Implementing and Analysing Financial Contracts on Blockchain. In: Financial Cryptography and Data Security (FC), Springer, Lecture Notes in Computer Science, vol 12063, pp 496–511
159. Yu XL, Al-Bataineh OI, Lo D, Roychoudhury A (2020) Smart Contract Repair. ACM Transactions on Software Engineering and Methodology 29(4):27:1–27:32
160. He J, Balunovic M, Ambroladze N, Tsankov P, Vechev MT (2019) Learning to Fuzz from Symbolic Execution with Application to Smart Contracts. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS), ACM, pp 531–548
161. Liu Y, Li Y, Lin S, Zhao R (2020) Towards automated verification of smart contract fairness. In: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), ACM, pp 666–677

Xuelian Cao received the B.S. degree in Computer Science and Technology from the Southwest University, Chongqing, China in 2018. She is currently studying for a master degree in Computer Application Technology at the Southwest University, Chongqing, China. She is interested in Blockchain technology and Game theory. Her mailing address is: No.2 Tiansheng Road, the Centre for Research and Innovation in Software Engineering, School of Computer and Information Science, Southwest University, Beibei, Chongqing 400715, China. Her email is xuelian7@email.swu.edu.cn.

Jianhui Zhang received the B.S. degree in information security from the Chongqing University, Chongqing, China in 2020. And now he is now studying for a master degree at Southwest University, Chongqing, China. He is now interested in Blockchain technology and Smart contract faults. His mailing address is: No.2 Tiansheng Road, the Centre for Research and Innovation in Software Engineering, School of Computer and Information Science, Southwest University, Beibei, Chongqing 400715, China. His email is 1uvu.zhang@gmail.com.

Xuechen Wu received the B.S. degree in computer science and technology from Beijing Foreign Studies University, Beijing, China in 2019. From 2019 to date, she is studying for a master degree in college of computer and information science of Southwest University, Chongqing, China. She is now interested in the Blockchain technology and smart contracts security. Her mailing address is: No.2 Tiansheng Road, the Centre for Research and Innovation in Software Engineering, School of Computer and Information Science, Southwest University, Beibei, Chongqing 400715, China. Her email is wxc227@email.swu.edu.cn.

Bo Liu received the B.S. degree in software engineering from the Wuhan University, Wuhan, China in 2004, and the Ph.D. degree in computer science and technology from Chongqing University, Chongqing, China in 2012. From 2013 to date, he is a Lecturer with the Department of Software Engineering of Southwest University, Chongqing, China. He is the author of more than 20 articles, and the PI or a member in more than 5 scientific projects on service computing, cyber physical systems, software system trustworthiness, and intelligent algorithms. He is now interested in Microservice and Blockchain systems. His mailing address is: No.2 Tiansheng Road, the Centre for Research and Innovation in Software Engineering, School of Computer and Information Science, Southwest University, Beibei, Chongqing 400715, China. His email is liubocq@swu.edu.cn, and and His homepage is http://computer.swu.edu.cn/s/computer/index2/20201201/4350569.html.